



Welcome to

AASD 4011: Deep Learning II

Credit hours: 48

Instructor: Dr. Moe Fadaee

Email: mo.fadaee@georgebrown.ca

George Brown College

Agenda:

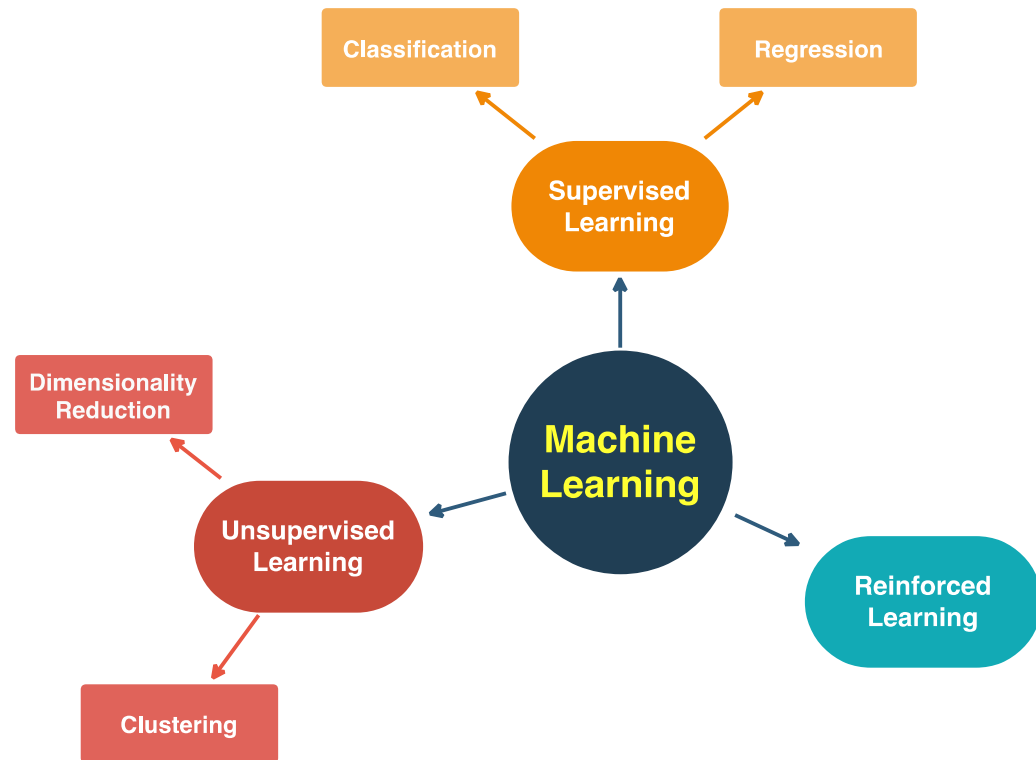
1. YOLO

Machine Learning subcategories:

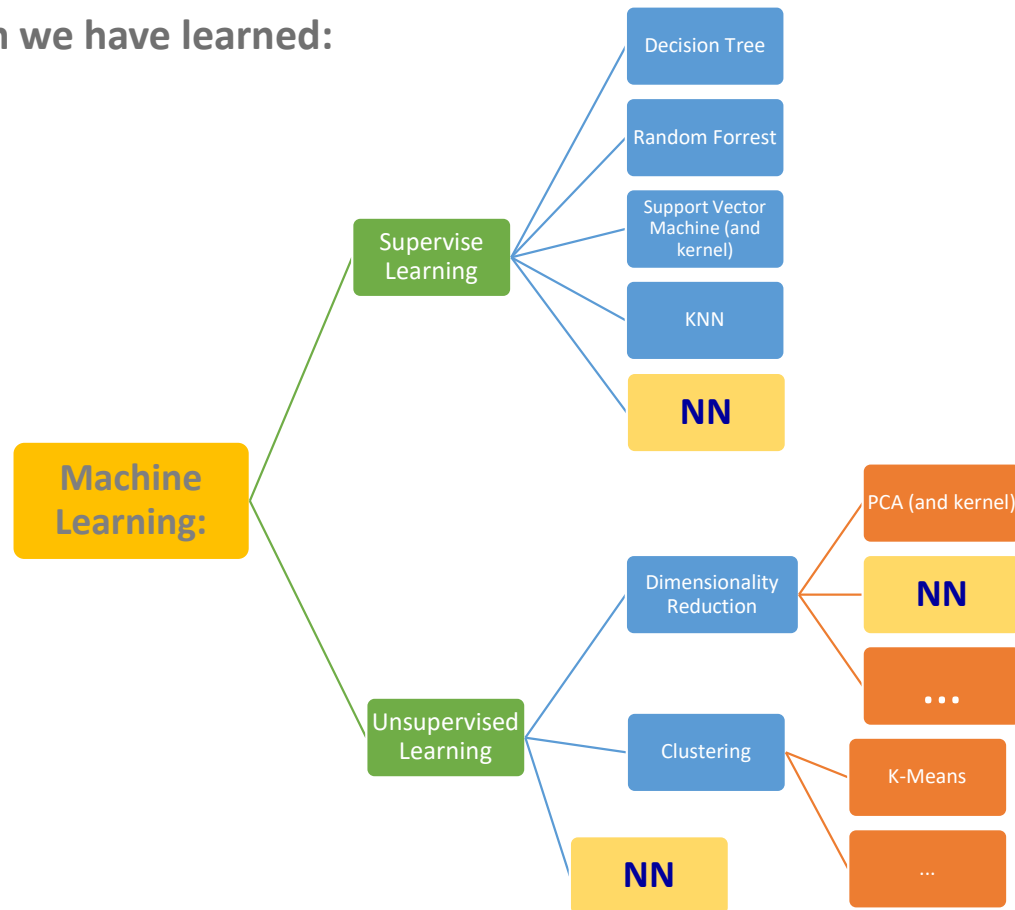
- I. Supervised Learning
- II. Un-supervised Learning
- III. Reinforced learning
- IV. Others...

Where is NN:

Almost all of them



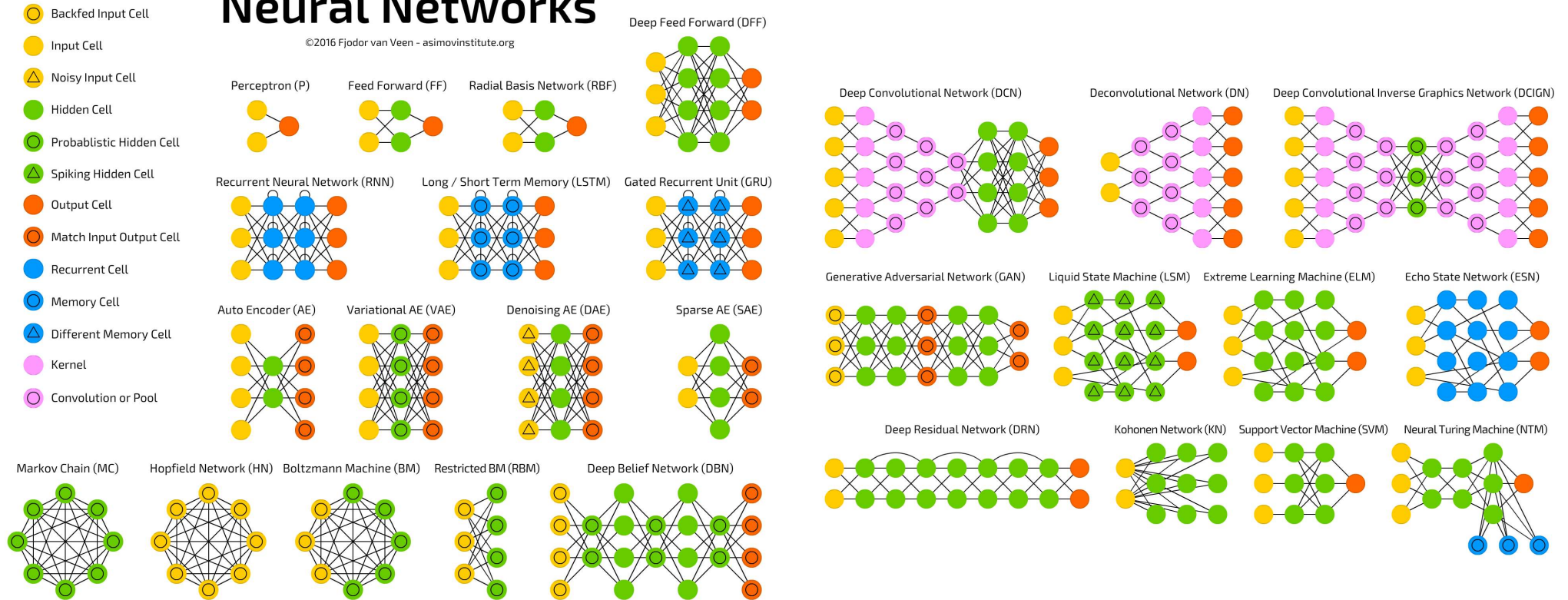
What algorithm we have learned:



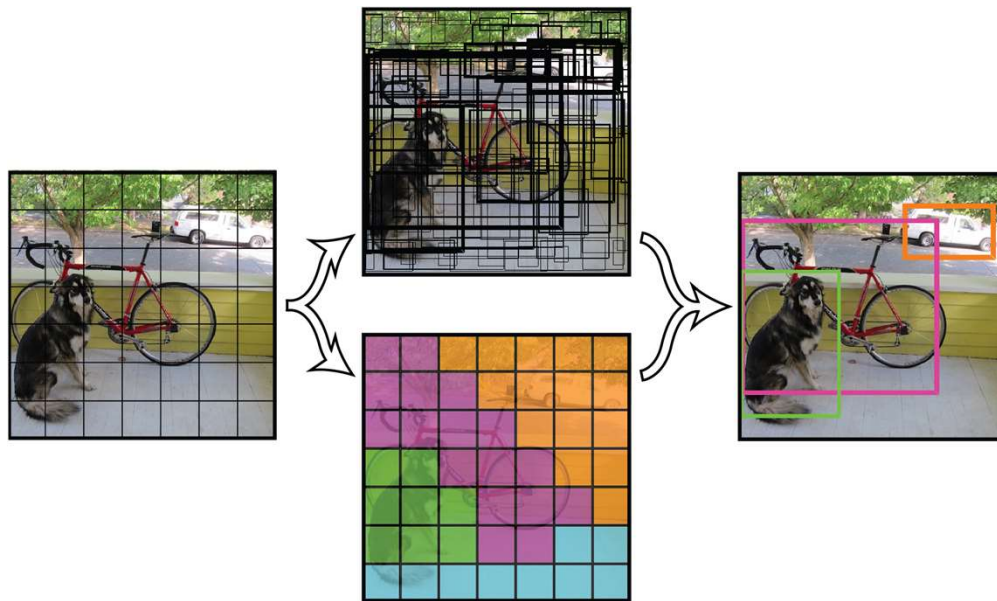
Different structures of Neural Networks:

A mostly complete chart of Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org

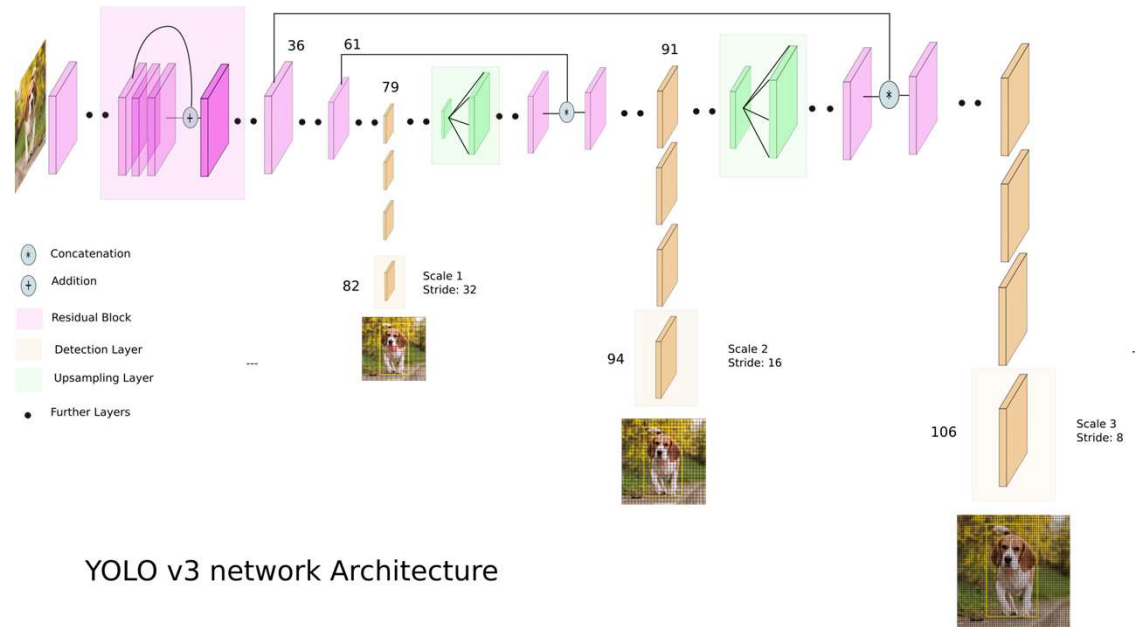


YOLO (You Only Look Once)

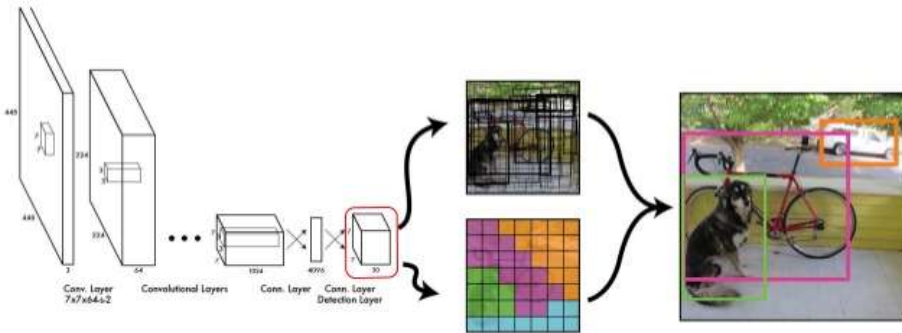


Object detections:

YOLO (You Only Look Once)



YOLO v3 network Architecture



Object detections: YOLO

How it works?

YOLO (You Only Look Once): YOLO is a very effective (real time) object detection algorithm first published by Joseph Redmon at 2015.

The original paper: You Only Look Once: Unified, Real-Time Object Detection

<https://arxiv.org/abs/1506.02640>

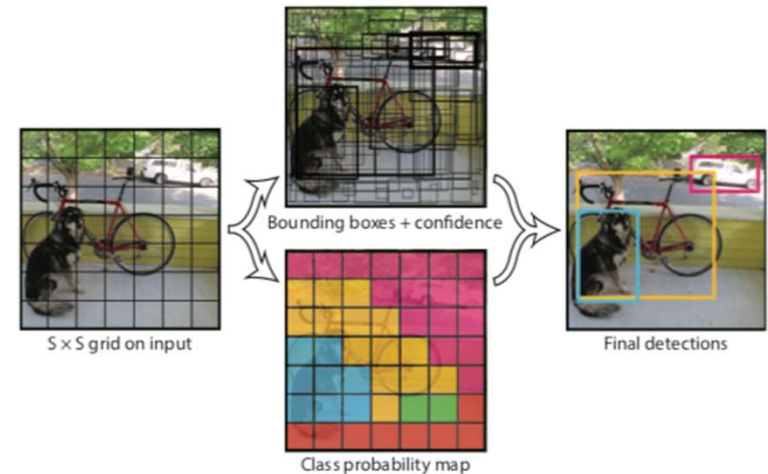


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

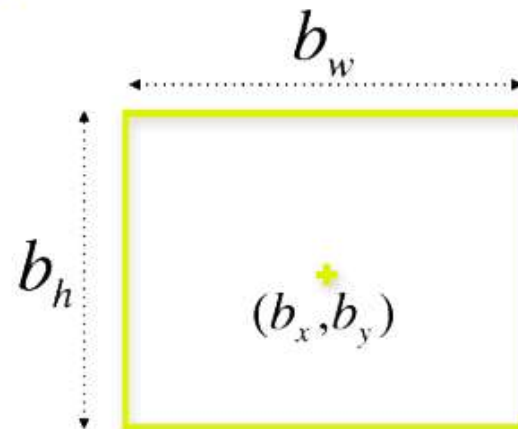
Object detections: YOLO

How it works?

What we are looking for in YOLO are:



$$y = (p_c, b_x, b_y, b_h, b_w, c)$$

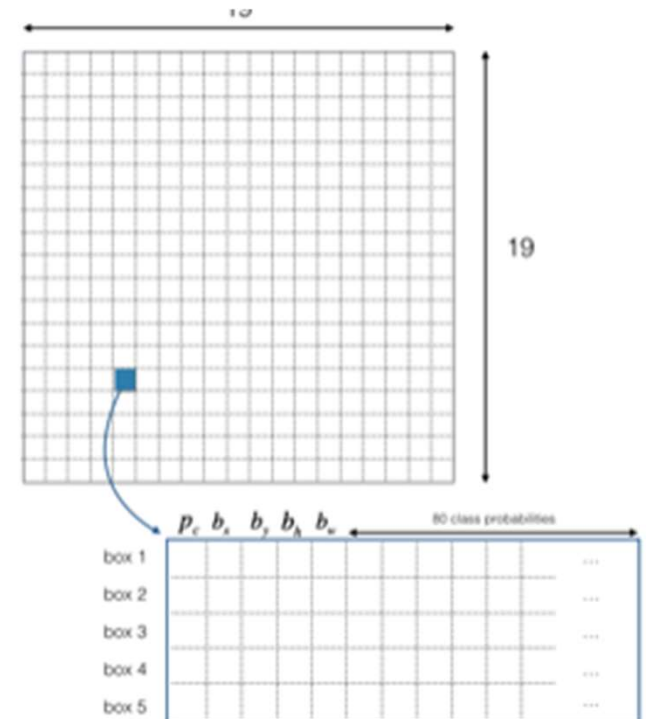
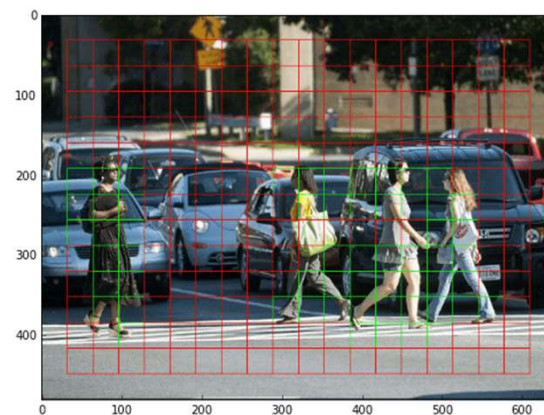
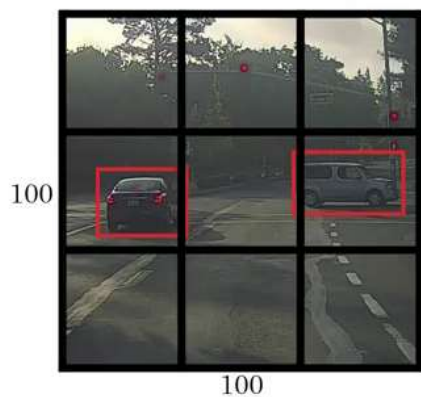


Object detections: YOLO

How it works?

As mentioned before, we are not trying to find ROI.

- YOLO read the input image and divide it into $S \times S$ grid
- Then for each grid cell predict B bounding box, box confidence, classes and probabilities



Object detections: YOLO

How it works?

Lets take a grid as an example, for each grid the following values will be estimated (assuming we have 3 classes):

y =	pc
	bx
	by
	bh
	bw
	c1
	c2
	c3

If there is no object in this grid the Pc is 0 and the rest of the parameters wont matter

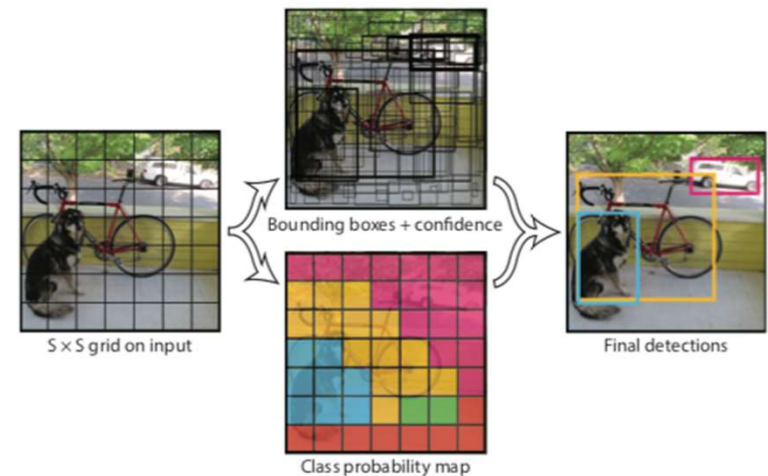


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

Object detections: YOLO

How it works?

If a grid contain an object then these variables are predicted. If we 16 grids and 3 classes, the output will be $4 \times 4 \times (B \times 5 + 3)$

Note that the bounding box are predicted with respect to each cell.

y =	pc
	bx
	by
	bh
	bw
	c1
	c2
	c3

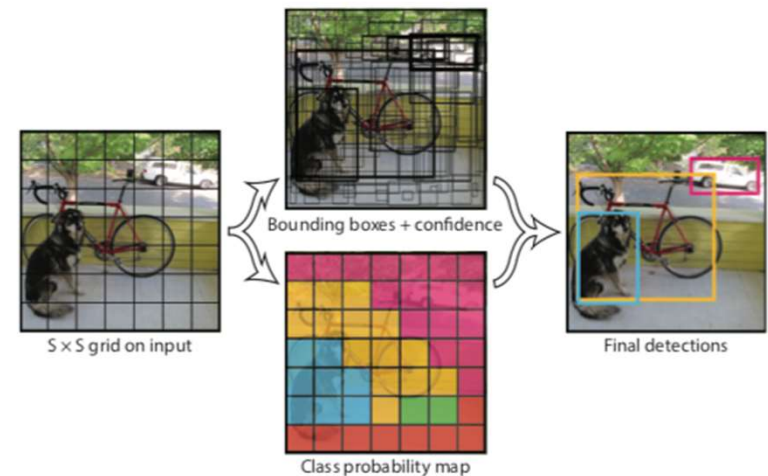


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B \times 5 + C)$ tensor.

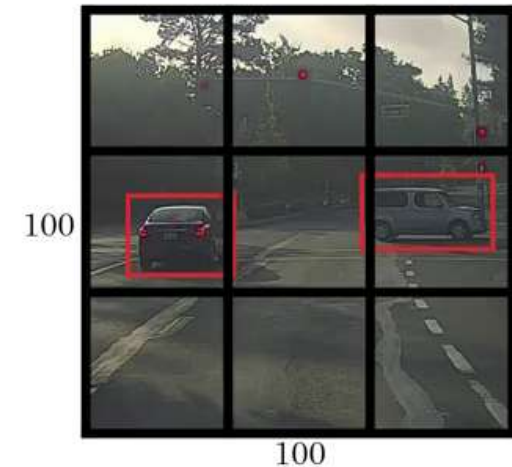
Object detections: YOLO

How it works?

b_x and b_y are the coordinate of the center point of the object wrt this grid (wrt the 0 and 0)

b_h is the height of the bounding box to the height of the grid (can be more than 1)

b_w is the width of the bounding box to the width of the grid (can be more than 1)



y =	pc
	b_x
	b_y
	b_h
	b_w
	c1
	c2
	c3

(0,0)



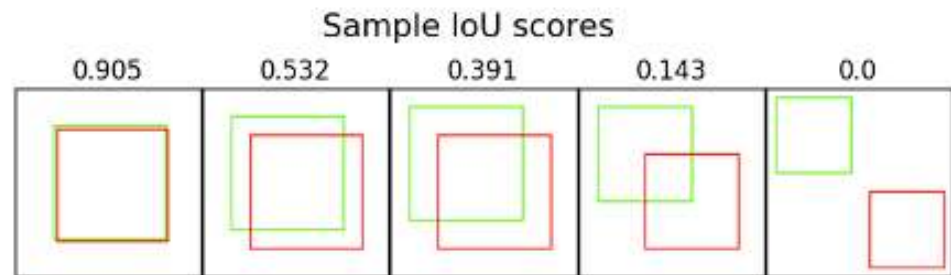
(1,1)

Object detections: YOLO

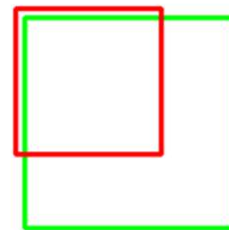
How it works?

Intersection over Union:

$\text{IoU} = \text{Area of the intersection} / \text{Area of the union}$

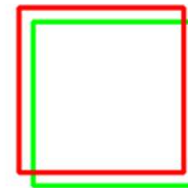


IoU: 0.4034



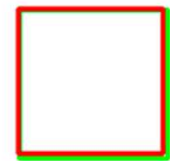
Poor

IoU: 0.7330



Good

IoU: 0.9264



Excellent

Object detections: YOLO

How it works?

Non-Max Suppression:

Objects detect multiple time, each time with a different probability



Object detections: YOLO

Performance

mAP: mean Average Precision

Average Precision is the area under the precision-recall curve. The mAP is the mean of the AP calculated for all the classes.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

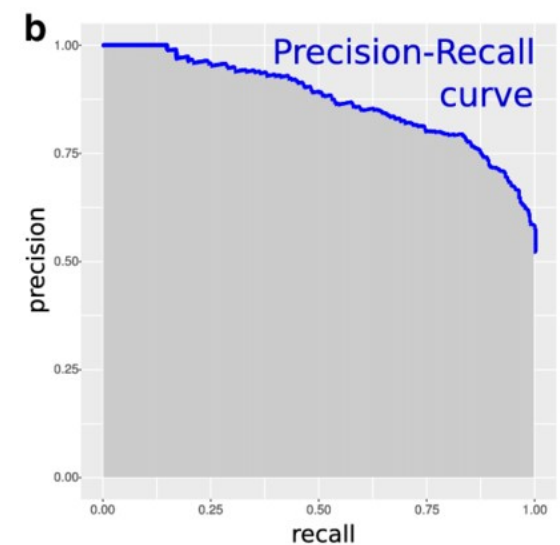
$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

TP = True positive

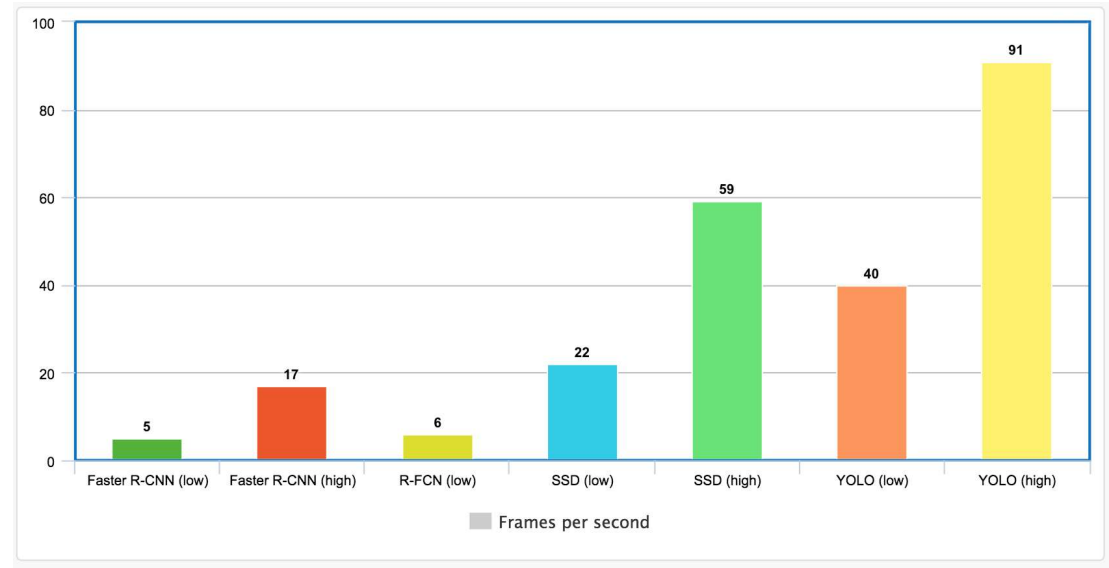
TN = True negative

FP = False positive

FN = False negative

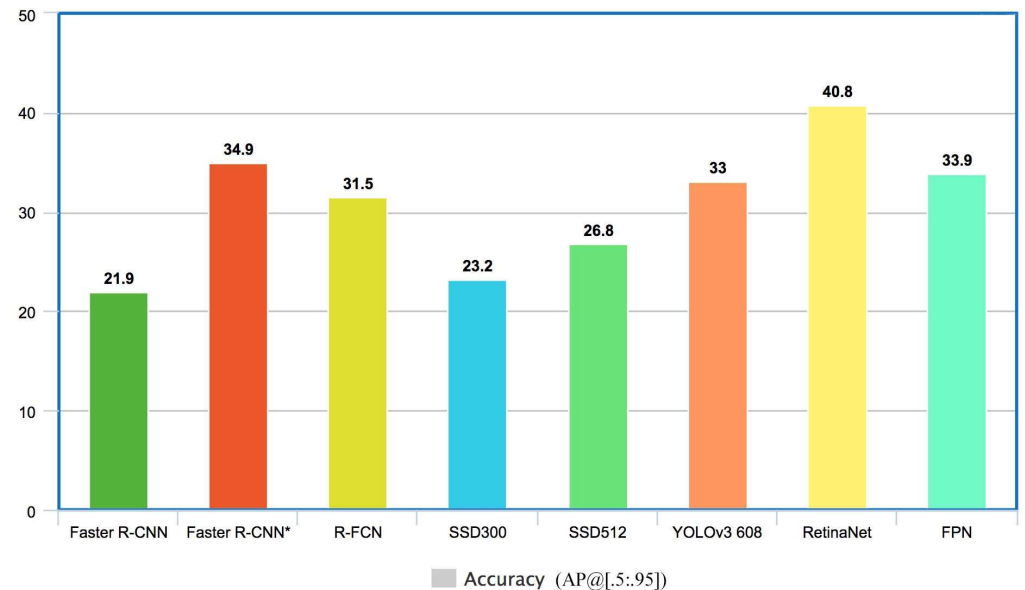


Object detections: Performance



Accuracy comparison:

<https://jonathan-hui.medium.com/object-detection-speed-and-accuracy-comparison-faster-r-cnn-r-fcn-ssd-and-yolo-5425656ae359>

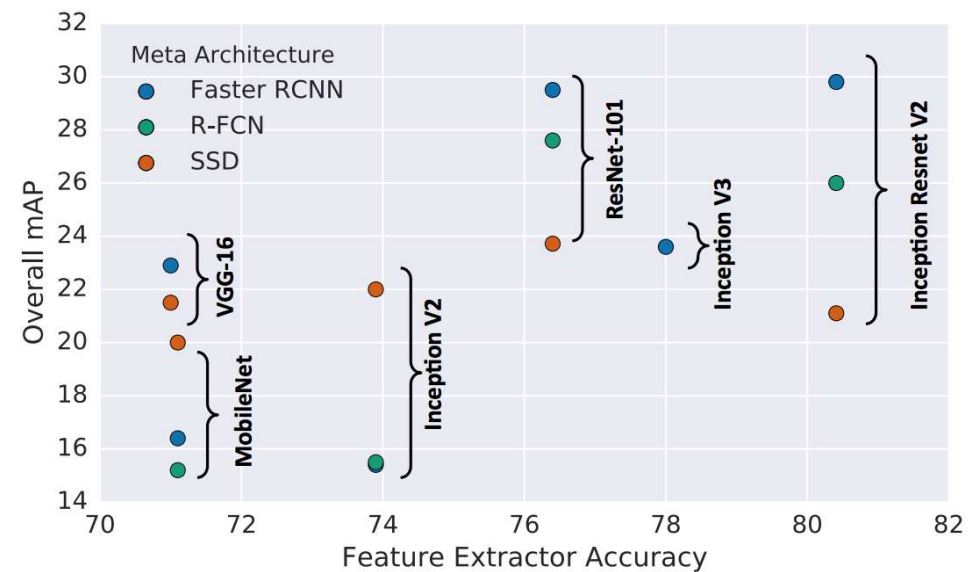
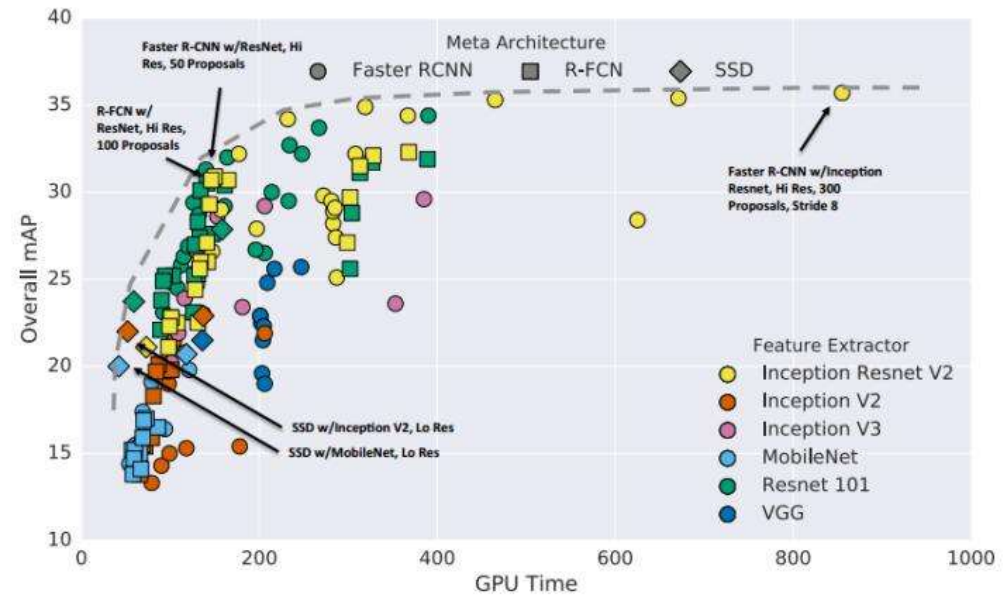


Object detections: Performance

Accuracy comparison:

<https://jonathan-hui.medium.com/object-detection-speed-and-accuracy-comparison-faster-r-cnn-r-fcn-ssd-and-yolo-5425656ae359>

<https://arxiv.org/pdf/1611.10012.pdf>



Object detections: Performance

Accuracy comparison:

<https://jonathan-hui.medium.com/object-detection-speed-and-accuracy-comparison-faster-r-cnn-r-fcn-ssd-and-yolo-5425656ae359>

<https://arxiv.org/pdf/1611.10012.pdf>

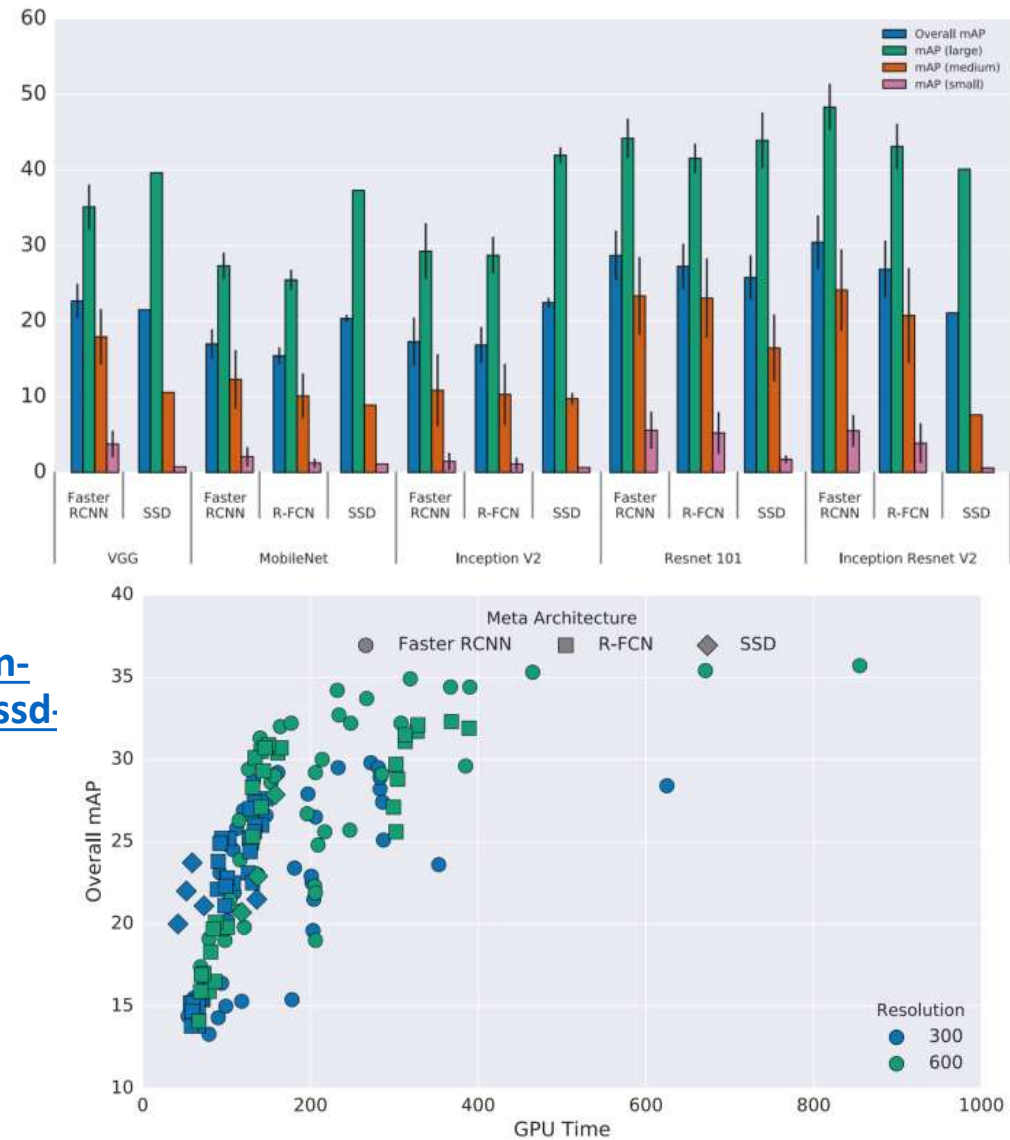
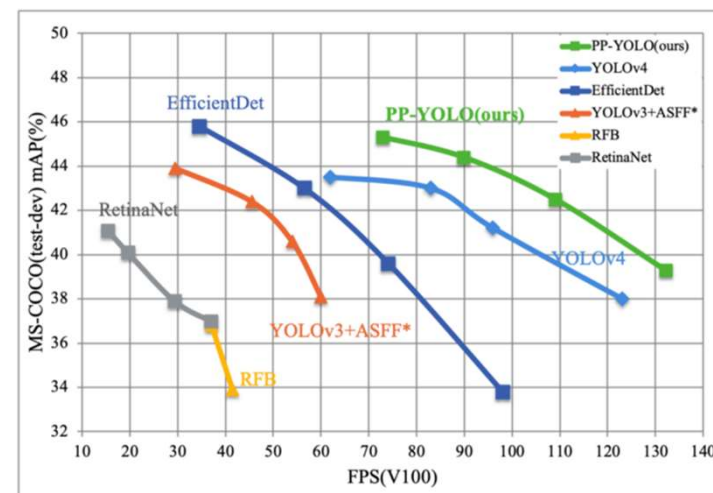
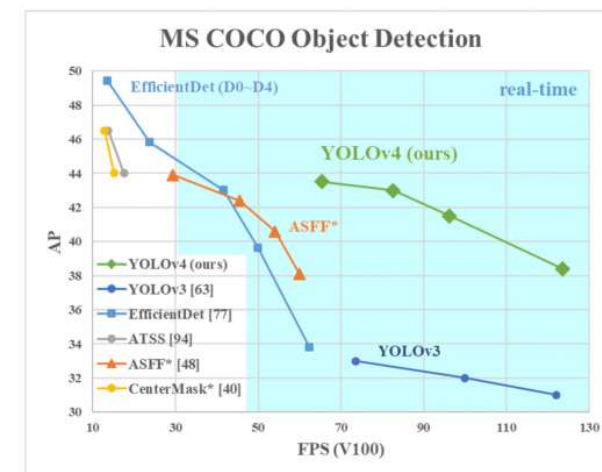


Figure 5: Effect of image resolution.

Object detections: Different YOLO versions,

1. YOLO v1, 2016
2. YOLO v2, Faster
3. YOLO v3, flexible, can speed an
4. YOLO V4, significantly improvements in the detector
5. YOLO V5, a Pytorch implementation, faster...
Controversy, A lot of heated discussion....
6. PP-YOLO

Model	Train	Test	mAP	FPS
YOLO	VOC 2007+2012	2007	63.4	45
Fast YOLO	VOC 2007+2012	2007	52.7	155
YOLOv2	VOC 2007+2012	2007	76.8	67
YOLOv2 544x544	VOC 2007+2012	2007	78.6	40
Tiny YOLO	VOC 2007+2012	2007	57.1	207
YOLOv2 608x608	COCO trainval	test-dev	48.1	40
Tiny YOLO	COCO trainval	-	-	200
YOLOv3-320	COCO trainval	test-dev	51.5	45
YOLOv3-416	COCO trainval	test-dev	55.3	35
YOLOv3-608	COCO trainval	test-dev	57.9	20
YOLOv3-tiny	COCO trainval	test-dev	33.1	220
YOLOv3-spp	COCO trainval	test-dev	60.6	20



Object detections: Performance Tuning

Different approaches to improve model performance

1. Add more images
2. Train for longer (more epoch)
3. Tune the architecture



Python practice1 :

Explore features with OpenCV

Build an object detection model using OpenCV and cvlib



Python practice 2: Using YOLO5 in COLAB

1. Prepare dataset

a) Download images

Use <https://github.com/hardikvasa/google-images-download> to download several images at once

Installation: <https://google-images-download.readthedocs.io/en/latest/installation.html>

Using pip: `pip install google_images_download`

Or use other tools like chrome extension called: **Download All Images**

b) Annotate/Label your images

Use the tools discussed before



Python practice 2: Using YOLO5 in COLAB

Colab Env. To train our model:

My Colab page (with vevent2020cns@gmail.com):

<https://colab.research.google.com/drive/1fEvcadO3mWeK5at6qDA63TKx93rSvrNp>

Roboflow:

Create a free account and upload your dataset or use one of public ones

Get your own custom model into Roboflow: <https://blog.roboflow.com/getting-started-with-roboflow/>

RoboFlow samples: <https://public.roboflow.com>



Python practice 2: Using YOLO5 in COLAB

Step by Step

1. Create an account in <https://public.roboflow.com>, and from public database, Fork one of the databases. (go to one of the database that you like, click on Fork on top right hand side)
2. After you Fork it, generate a link with the version (Yolo v5 PyTorch), again click on “Generate” on top right and give it a name, and select Yolo v5 PyTorch format and “show download code” option

This will give you a link like this:

```
!curl -L "https://app.roboflow.com/ds/3iTLCiusC1?key=jtd5z6ZMsE" > roboflow.zip; unzip roboflow.zip; rm roboflow.zip
```

3. We are going to follow: <https://blog.roboflow.com/how-to-train-yolov5-on-a-custom-dataset/>:

<https://colab.research.google.com/drive/1gDZ2xcTOgR39tGGs-EZ6i3RTs16wmzZQ>

and click on “Copy To Drive”, you should sign in to your google account,

This will open a new tab for you to work on

Other Roboflow guides on different models:

<https://models.roboflow.com>